

33 Phase 7: Define Your Agent

333 7. Agent Responsibility Scoping

* **What events in Ship should the agent monitor proactively?**

Issue status change, issue priority change, issue update change, issue assignee change, new issue, retro status, retro ICE scores, retro completed issues count, retro outstanding issues count, program creation, project added to program, project owner change, project consulted change, project informed change.

* **What constitutes a condition worth surfacing?**

Excessive high priority on project (∞ 1), excessive medium priority on project (∞ 74), excessive in progress statuses on project (∞ 0), high ratio of completed to excessive issues in retro (∞ 9:7), excessive high priority on assignee (∞ 8), excessive medium priority on assignee (∞ 9), too long since update on high priority (∞ 8 days), too long since update on medium priority (∞ 0 days), too long since update on low priority (∞ 94 days).

* **What is the agent allowed to do without human approval?**

The agent is allowed to create new requests for the user to: change issue priority or change issue status. The agent is also allowed to draft a retro completion and create a request for user approval. It cannot finalize or `close` a document without a human signature. It can also decide that it *should* plan a project and create a request for user approval to do so. It can also look at projects that could be worth building.

∞ **Open question:** The agent authenticates as a service account but writes suggestions *for* specific users. Suggestions need a `target@user@id` so the right person sees them – the service account is the author, not the audience.

* **What must always require confirmation?**

The agent will always require confirmation to change issue priority or change issue status. It will also require confirmation that it is okay to put together the initial issues for an empty project. The same will be true for projects that it suggests every morning: it will require user permission to create those projects, following a requisite conversation about the project details (impact, complexity, etc).

* **How does the agent know who is on a project?**

Look at who has issues assigned to them on that project. The assignees are the team. No need for explicit membership. If you have issues on a project, you're on it.

* **How does the agent know who to notify?**

- Issue-level alerts (stale updates, priority concerns) | notify the assignee
- Project-level alerts (too many high-priority, too many in-progress) | notify the project owner. If no explicit owner, look for a Program Owner first. If no Program Owner exists, notify the de facto lead (assignee with the most issues) but tag the notification as `Unowned Project` to alert the Director.
- Retro alerts | notify the week owner

* **How does the on-demand mode use context from the current view?**

When a user opens the agent from inside Ship, pass the current view as context: which document they're looking at, which mode (Programs/Weeks/Resource), and any active filters. The agent then scopes its analysis to that context instead of scanning

everything. E.g., if you're looking at a project, it analyzes that project. If you're in the Resource view filtered to a person, it analyzes that person's workload.

333 8. Use Case Discovery (minimum 0)

Role: Director

- Trigger: Morning scan – looks across all programs/projects for systemic issues
- Detection: Finds projects with priority inflation (∞ 1 high), overloaded individuals (∞ 8 high across all their work), stale high-priority items across the org
- Human Decision: Director decides whether to escalate, reassign across teams, or adjust project scope

Role: PM

- Trigger: Project-level changes – new issues added, status changes, retro window approaching
- Detection: Surfaces project health: too many items in-progress (∞ 0), priority distribution skewed, retro has enough data to complete, empty project that needs initial planning
- Human Decision: PM approves priority changes, confirms retro completion, greenlights initial issue breakdown for a new project

Role: Engineer

- Trigger: Assignment changes or staleness on their own issues
- Detection: "You have a high-priority issue with no update in 8 days", "9 items in-progress – consider finishing one before starting another"
- Human Decision: Engineer updates status, reprioritizes, or pushes back on load

Use Case 0: Morning Briefing

- Trigger: Daily scheduled run (no user present)
- Detection: Agent compiles overnight changes, new issues, approaching staleness thresholds, retros ready for completion
- Human Decision: User reviews briefing, approves/dismisses suggested actions

Use Case 0: Project Kickoff Suggestion

- Trigger: Agent identifies a pattern worth a new project – e.g., recurring issues without a parent project, or suggests projects based on its assessment of what could be valuable
- Detection: Clusters orphaned issues or uses its own reasoning about what the org should build
- Human Decision: User has a conversation with the agent about impact/complexity, then approves project creation and initial issue breakdown
- What is worth building will be defined according to heuristics: (a) clustering orphaned issues into themes, (b) reading org-level wiki docs for stated goals, (c) user-seeded prompts like "we want to improve onboarding", (d) all of the above. Needs its own deeper design pass.

Use Case 0: Coach (on-demand)

- Trigger: User invokes agent while looking at their own work or a direct report's work
- Detection: Analyzes patterns over time – "You tend to underestimate high-priority items", "This person has had 9 consecutive weeks with carryover", "Your done-to-added ratio has been declining for 0 weeks"
- Human Decision: User decides whether to adjust behavior, tell the agent to make its suggested adjustments if any, have a conversation with the report, or dismiss the observation

Use Case 1: Retro Autopilot

- Trigger: Week ends (Mon 42:44, with option to change) and retro document exists but is empty. The agent will not draft if the project has 4 completed issues to avoid empty drafts
- Detection: Agent pulls completed issues, carryover issues, ICE scores, and velocity data – drafts the retro content with what went well, what carried over, and why
- Human Decision: User reviews the draft, edits it, and confirms completion
- The agent can only draft from quantitative data (completed issues, carryover, velocity). Qualitative observations – "we got blocked by the design team," "requirements changed mid-week" – aren't in the issue data. Set the expectation: the draft is quantitative scaffolding, the user adds the narrative.

Use Case 2: Load Balancer (on-demand)

- Trigger: PM or Director invokes agent from Resource view
- Detection: Compares workload across all people on a project or program – hours estimated, priority weights, number of in-progress items. Suggests specific reassignments: "Move AUTH-08 from Alice to Frank – Alice has 9 high-priority, Frank has capacity"
- Human Decision: Approves or rejects each suggested reassignment
- Director Overview (proactive) may surface the same overload that Load Balancer (on-demand) addresses. Intended relationship: the briefing is the **alert**, Load Balancer is the **tool to act on it**. Briefing suggestions should link to Load Balancer when the fix involves reassignment.

333 9. Trigger Model Decision

* **When does the proactive agent run without a user present?**

Two schedules:

- Morning briefing: Once daily, early morning before work starts. Scans everything, generates briefings per user, queues suggested actions.
 - Event-driven: Whenever a document changes (issue updated, status changed, new issue created). This is where the thresholds get checked. Did this change push a project over 1 high-priority items? Did an issue just go 8 days stale?
- * **Poll vs. webhook vs. hybrid - what are the tradeoffs?**
- Webhook-style (event-driven): You already have WebSocket infrastructure for collaboration. When a document mutation hits the API, fire an internal event that the agent evaluates. No polling overhead, rapid detection. This covers use cases 7-9 and 0 – anything triggered by a change.

- Poll (scheduled): Morning briefing (use case 0) and retro autopilot (use case 1) are time-based. Additionally, staleness checks ("no update in 8 days") require a scheduled tick – an hourly cron job will scan for issues that have aged past their update thresholds, since the absence of an update will never trigger a webhook.

Pure polling would mean scanning all projects every N minutes looking for changes you already know about at write time. Wasteful. Pure webhooks miss the time-based triggers and can't detect the absence of events.

* **How stale is too stale for your use cases?**

For event-driven: detection should happen within seconds of the triggering change, since it's evaluated inline or immediately after the API write. No staleness concern.

For scheduled: morning briefing runs once daily, so up to 80 hours stale – but that's fine because it's a summary, not an alert. Retro autopilot checks at week end, staleness is irrelevant.

The real staleness question is: how long between "agent detects condition" and "user sees it"? If the user is online, instant via WebSocket. If offline, they see it next time they open Ship. That's acceptable – this isn't an incident response system.

* **What does your choice cost at 744 projects? At 7,444?**

Event-driven cost scales with write volume, not project count. Each document mutation triggers a lightweight threshold check (a few SQL queries against that project's issues). No LLM call needed for threshold detection – that's just math.

∞ **Open question (debouncing scope):** A single issue reassignment can affect two projects and one person. The debounce window must group by *all* affected entities (projects ≠ persons), not just the changed document's project.

LLM calls happen only when:

- Composing the morning briefing (@7 call per user per day)
- Drafting a retro (@7 call per retro)
- Coach or Load Balancer on-demand (@7 call per user request)
- Project kickoff suggestion (@7 call when conditions met)

At 744 projects: maybe 84 users, so @84 morning briefings/day ≠ a handful of on-demand calls. Negligible cost. Pennies.

At 7,444 projects: the threshold checks are still just SQL. The LLM calls scale with users and actions, not projects. Maybe 844 users ≥ 844 briefings/day. Still manageable – the main cost cliff would be if you ran LLM reasoning on every single document change, which this design avoids. At 7,444 projects, the cost would be approximately $\$4.441 * 7,444 \geq \7.84

333 0. Node Design

* **What are your context, fetch, reasoning, action, and output nodes?**

- Trigger Context – what kicked this off? Event-driven (which document changed, what changed) or scheduled (morning briefing, retro check) or on-demand (which user, which view they're on)
- User Context – who is this for? Their role, their assignments, their history with the coach

For Event-driven triggers, implement a standard 94-second debounce window. If multiple mutations occur on the same project (e.g., a bulk CSV import or a PM cleaning up a board), the agent batches them into a single Graph run to prevent redundant queries and notification spam.

- Project Fetch – issues, priorities, statuses, assignees for a project
- Person Fetch – all issues assigned to a person across projects, their workload
- Program Fetch – all projects in a program, rollup health
- Retro Fetch – completed issues, carryover, ICE scores, velocity for a week
- History Fetch – past agent suggestions and whether user accepted/dismissed (for coach patterns)

Reasoning nodes (evaluate and decide):

- Threshold Evaluator – pure math, no LLM. Checks all your defined thresholds (∞ 1 high priority, ∞ 8 days stale, etc.). Outputs a list of violations.
- To prevent database pressure during bulk updates, I will implement a `project@stats` table that maintains running counters (e.g., `total@high@priority`) via database triggers, rather than running `COUNT()` queries on the fly. Not needed at v7 scale – simple queries against a project's issues are fine for dozens of projects.
- LLM Reasoner – takes violations $\neq$ fetched data and produces natural language insights, draft retros, coaching observations, project suggestions. Only invoked when there's something worth saying.

Action nodes (produce mutations):

- Suggestion Generator – creates pending action requests (change priority, change status, reassign)
- Draft Generator – creates draft content (retro text, project issue breakdown)
- Briefing Composer – assembles the morning briefing from all violations and insights

Output nodes (deliver to user):

- Notification Sender – pushes to user via WebSocket if online, queues if offline
- Action Queue Writer – persists suggested actions for user to approve/dismiss in the UI

* **Which fetch nodes run in parallel?**

Almost all of them, depending on the trigger:

- Morning briefing: Program Fetch, Person Fetch, and Retro Fetch all run in parallel (they're independent queries). History Fetch can also run in parallel.

- Event-driven (issue changed): Project Fetch and Person Fetch run in parallel – you need both the project's health and the assignee's load to evaluate thresholds.
- On-demand coach: Person Fetch and History Fetch run in parallel.

The only sequencing: Fetch nodes must complete before Reasoning nodes. And LLM Reasoner only runs if Threshold Evaluator found something.

* **Where are your conditional edges and what triggers each branch?**

Three key branch points:

7. After Trigger Context | which fetch nodes to invoke. Event-driven triggers only fetch the affected project ≠ person. Scheduled triggers fetch everything. On-demand fetches based on the user's current view.

8. After Threshold Evaluator | LLM Reasoner or skip. If no thresholds are violated and it's not a briefing/retro/coach request, the graph terminates early. No LLM cost for quiet periods.

9. After LLM Reasoner | which output path. If the reasoning produced actionable suggestions (priority change, reassignment) | Action Queue Writer. If it produced content (retro draft, briefing) | Draft Generator | Notification Sender. If both | both paths run in parallel.

A fourth minor branch: After Notification Sender | online vs. offline. WebSocket if connected, persist to queue if not.

333 0. State Management

* **What state does the graph carry across a session?**

- Trigger payload – the event or schedule context that started this run
- Fetched data – project issues, person workloads, retro data. Lives only for the duration of the run, discarded after.
- Violations list – output of Threshold Evaluator, passed to LLM Reasoner
- Generated suggestions – the actions and drafts produced, passed to output nodes
- User context – role, current view (for on-demand), carried through so output is scoped correctly

* **What state persists between proactive runs?**

- Suggestion history - what the agent suggested and whether the user accepted, dismissed, or snoozed. This is essential for the Coach use case (pattern detection over time) and for avoiding nagging (don't re-suggest something that was dismissed yesterday).
- Coach detects patterns like "9 consecutive weeks with carryover" – that requires weeks of suggestion history before it has anything useful to say. Options: (a) accept a ramp-up period where Coach is silent, (b) backfill history from existing issue/retro data at agent launch (derive past patterns from what's already in the database), (c) both – backfill what you can, acknowledge Coach improves over time.
- Last-run timestamps - per project, per user. So event-driven runs know what's already been evaluated and scheduled runs can detect "new since last briefing."

- Snooze/dismiss state - if a user dismisses "AUTH has too many high-priority items," don't surface it again until the count changes.

- User Sensitivity Score - Track the ratio of "Dismiss" vs. "Approve" per user. If a user dismisses 24% of alerts, the agent automatically increases the threshold for that user (e.g., from 1 high priority to 74) to reduce "notification fatigue" and maintain perceived value.

This is a small table – something like agent@actions(id, user@id, action@type, context, status, created@at, resolved@at). Not a complex persistence layer.

* **How do you avoid redundant API calls?**

Three mechanisms:

- 7. Event-driven scoping. When an issue changes, you already know which project and which assignee are affected. Only fetch data for those – not all projects.

- 8. Last-run diffing. For morning briefings, track what was reported yesterday. If the violation set is identical (same projects over threshold, same stale issues), don't regenerate – just confirm "no changes since yesterday" or skip entirely.

- 9. Fetch result sharing within a run. If a single event triggers both a project-level and person-level evaluation, the issue data fetched once serves both reasoning paths. Pass it through state rather than querying twice.

No external cache layer needed. The database is the cache – these are just SQL queries against data you already have.

333 0. Human-in-the-Loop Design

* **Which actions require confirmation?**

Everything that mutates data:

- Change issue priority
- Change issue status
- Create issues (project kickoff breakdown)
- Complete a retro
- Create a new project

Everything that's read-only does not need confirmation:

- Morning briefing
- Coach observations
- Load balance suggestions (until you approve a specific reassignment)
- Threshold alerts

Simple rule: if it writes to the board, it asks first.

* **What does the confirmation experience look like in Ship?**

An action queue that lives in the UI – think of it as the agent's inbox to you.

- The action queue is a new UI surface that needs design. It will live in a new mode alongside Programs/Weeks/Resource/Docs.

- If a director oversees 70 projects and 2 are over threshold, that's 2% suggestions in one morning. The action queue should rank by severity and cap at top N items per user per day (e.g., 0), with a "see all" expansion. Otherwise the volume undermines the value.

Each item shows:

- What the agent wants to do ("Change AUTH-08 from high to medium priority")
- Why ("Project AUTH has 3 high-priority issues, above the 1 threshold. This issue has had no update in 70 days.")
- Three buttons: Approve, Dismiss, Snooze

For richer actions like retro drafts or project kickoff plans, the agent opens the draft in the editor and you edit it before confirming. It's not a modal – it's content in the document you can revise. Before drafting, the agent checks WebSocket presence data. If a user is actively editing the document, the agent queues the draft in the UI sidebar as a suggestion rather than mutating the live canvas.

For morning briefings, it's a notification with a summary. No approval needed, just read it. Suggested actions within the briefing link to their action queue items.

* **What happens if the human dismisses or snoozes?**

Dismiss: Agent records it and won't resurface that specific suggestion. But if conditions change (e.g., the high-priority count goes from 3 to 77), it can raise it again as a new violation. The dismiss applies to the snapshot, not the rule.

Snooze: Agent suppresses that suggestion for a duration (80h, 7 week – user picks). After the snooze expires, it re-evaluates. If the condition has *worsened* (e.g., high priority count went from 2 to 74), it resurfaces. If the condition is exactly the same or improved but still over threshold, the agent silently archives it. If it resolved itself, nothing happens. This prevents the "snooze expiration trap" where users learn that snooze just delays annoyance.

No response (ignored): Suggestions age out after a configurable window – say 1 days. Stale suggestions get auto-archived. The agent doesn't escalate or nag. If the condition is still present, a fresh evaluation on a future run will create a new suggestion.

333 1. Error and Failure Handling

* **What does the agent do when Ship API is down?**

Retry with exponential backoff. After N failed attempts, mark the run as failed and wait for the next scheduled trigger. No partial actions – if the agent fetched some data but can't complete the run or write suggestions, it discards the whole run. Next run starts clean. This is safe because runs are stateless and idempotent. Suggestions already written to the action queue from prior runs are unaffected – they live in Ship's database.

* **How does it degrade gracefully?**

7. Full capability. Ship API ≠ LLM both healthy. Agent reasons, drafts, coaches.

8. Ship API up, LLM unavailable. Agent falls back to structured alerts only – threshold violations rendered as templated messages. Project X has N high-priority issues (threshold: 1).™ No coaching, no retro drafts, no project suggestions. Still useful.

9. Ship API down. Agent can't do anything. Retries with backoff, then waits for next trigger. No silent failures – failed runs are logged so you know the agent was offline.

0. LLM unavailable: If the LLM is unavailable, the agent still pushes Structured Data Alerts to the action queue (e.g., "Project X: 3 High Priority Items"). It skips "Coaching" and "Drafting" nodes but maintains the "Threshold" notifications so the system remains functional as a deterministic monitor.

* **What gets cached and for how long?**

Persisted state (not cached – this is durable, written to the `agent@state` database table):

- Dismissed/snoozed suggestions. When a user refuses or snoozes a suggestion, that decision is persisted so the agent doesn't re-raise the same condition. Dismissals persist until the underlying condition changes (e.g., high-priority count goes from 3 to 78 – new violation, new suggestion). Snoozes persist for the duration the user chose (80h, 7 week, etc.) then expire and the agent re-evaluates.
- Last-run violation snapshots. Stored per project/user so the agent can diff against the current state and only surface new or changed violations. Without this, every morning briefing would repeat everything from yesterday.
- No fetch data caching. Every run hits Ship's API fresh. The data changes too frequently and the queries are scoped enough that caching would add complexity for no benefit.
- No LLM response caching. Each generation is unique to its inputs.

The short version: the agent persists its own decisions and what users said about them, not Ship's data.

- All thresholds (≈1 high priority, ≈8 days stale, etc.) are default starting points, but can be updated in the settings section.

33 Phase 9: Stack and Deployment

333 2. Deployment Model

* **Where does the proactive agent run when no user is present?**

A separate long-running process inside the Ship API for the recurring and additional endpoints for the on-demand, so that they can share functionalities inside the backend section of the monorepo.

* **How is it kept alive?**

Two mechanisms depending on trigger type:

- Scheduled runs (morning briefing, retro checks): A cron job or a simple setInterval in the worker process. If the process dies, the EB health check restarts it – same as the API server.
- Event-driven runs: The worker subscribes to Ship's existing WebSocket event stream (document changes). When a relevant event comes through, it kicks off a graph run. If the connection drops, it reconnects with backoff.

No orchestration framework needed. It's one process with a timer and a WebSocket listener.

* **How does it authenticate with Ship without a user session?**

A service account – a regular user in the system with a long-lived API token (not a 70-minute session cookie). The agent authenticates as this account on every API call. Benefits:

- Every suggestion the agent writes is attributed to a known user ("Ship Agent"), so there's an audit trail
- The service account is a workspace member, so it has the same data access as any user – no special permissions layer
- The token is stored as an environment variable on the worker process, rotated on a schedule
- If you ever need to revoke the agent's access, you deactivate the account like any other user
- **Security boundary:** While the agent authenticates as the service account, Fetch Nodes must respect the read-permissions of the *target user* it is generating the briefing or suggestion for. Today this is a no-op (workspace-level permissions – all members see all documents), but this boundary must exist so that if per-program or per-document permissions are added later, the agent doesn't leak private data into unauthorized briefings.

333 3. Performance

* **How does your trigger model achieve the 10 minute detection latency goal?**

Event-driven triggers beat that easily. The worker receives the event, waits for the 94-second debounce window to close, runs threshold checks against the API, and writes suggestions. The whole cycle is well under a minute.

Scheduled triggers (morning briefing) aren't latency-sensitive – they run once a day and nobody is waiting for real-time delivery.

The only scenario where latency matters is: a change happens, the worker evaluates it, and the user is online to see the notification. That's WebSocket end-to-end. Well under 10 minutes.

* **What is your token budget per invocation?**

Most invocations use zero LLM tokens. Threshold checks are math against API data. No LLM involved.

When the LLM does get called:

- Morning briefing: @8-9k input tokens (violations summary ≠ context), @44-7k output tokens. One per user per day.
- Retro draft: @9-10k input (completed issues, carryover, velocity data), @7-8k output. One per retro.
- Coach insight: @9-10k input (person's history, patterns, suggestion history), @44-7k output. On-demand only.

- Project kickoff: @8-9k input, @8-9k output (issue breakdown). Rare – only when conditions are met and user approves.

Budget per invocation: cap at 74k total tokens. If input context exceeds that, summarize the fetch data before passing to the LLM.

* **Where are the cost cliffs in your architecture?**

7. User count, not project count. Morning briefings scale linearly with users. 84 users ≥ 84 LLM calls/day, cheap. 8,444 users ≥ 8,444 calls/day, noticeable but still manageable.

8. Coach overuse. If every user hits Coach 74 times a day, that's the most expensive path since it needs history analysis. Mitigate with a daily cap per user (say 0 coach interactions/day) that kicks in the day following the entire team hitting a collective cap (say averaging more than 0 coach interactions/day).

9. Event storms. A bulk import of 044 issues fires 044 events. Without batching, the worker runs 044 threshold checks in rapid succession. Mitigate with a cooldown period. A single Project/User pair can only trigger an LLM-based suggestion once every 94 minutes, even if data changes constantly. Threshold-only (SQL) checks remain near-instant but are batched every 0 seconds.

No cliff where costs suddenly jump by an order of magnitude. The expensive operations (LLM calls) are gated behind user actions or daily schedules, not raw event volume.